

DBMON-99: Best Practice Summary

Series: DBMON — Database Monitoring | **Notebook:** 7 of 7 | **Created:** March 2026 | **Last Updated:** 09/18/2026

Overview

This notebook consolidates every actionable best practice for Dynatrace database monitoring extracted from the DBMON series (notebooks 01 through 06). It covers SQL databases, NoSQL databases, caches, message brokers, query analysis, dashboards, and alerting. Each best practice is definitive: it specifies exactly what to configure, what threshold to set, and what priority to assign.

Table of Contents

1. [OneAgent Instrumentation](#)
 2. [ActiveGate Extensions](#)
 3. [Span Attribute Coverage](#)
 4. [SQL Database Monitoring](#)
 5. [NoSQL Database Monitoring](#)
 6. [Cache Monitoring \(Redis / Memcached\)](#)
 7. [Message Broker Monitoring \(Kafka / RabbitMQ\)](#)
 8. [Search Engine Monitoring \(Elasticsearch\)](#)
 9. [Query Analysis and Optimization](#)
 10. [Dashboards and KPIs](#)
 11. [Alerting Thresholds](#)
 12. [SLO Definitions](#)
 13. [DQL Query Standards](#)
 14. [Where to Go Deeper — Topic Series Map](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
OneAgent	Deployed on all application hosts making database, cache, and messaging calls
ActiveGate	Environment ActiveGate for Extensions 2.0 (remote DB metrics), or — SQL extensions only, Dynatrace 1.346+ (staged rollout; verify it has reached your tenant) — SQL Extension Executor on Kubernetes via Dynatrace Operator 1.8+
Permissions	<code>storage:spans:read</code> , <code>storage:entities:read</code> , <code>storage:metrics:read</code>
Prior Reading	DBMON-01 through DBMON-06 for full context

1. OneAgent Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Deploy OneAgent on every host that makes database calls	OneAgent installed on all application servers, not just database servers	Critical	Deployment
2	Enable deep code-level instrumentation	OneAgent auto-instruments JDBC, ADO.NET, and native database drivers by default; do not disable	Critical	Deployment
3	Verify <code>db.system</code> attribute is populated	Run <code>fetch spans, from:-1h \filter isNotNull(db.system) \summarize count(), by:{db.system}</code> — every expected technology must appear	Critical	Validation
4	Verify <code>db.query.text</code> capture is enabled	Confirm normalized SQL/commands appear in <code>db.query.text</code> field; if blank, check OneAgent deep monitoring settings	Critical	Validation
5	Confirm <code>db.namespace</code> is present	Run <code>filter isNotNull(db.namespace)</code> on spans; missing values indicate driver-level gaps	Recommended	Validation
6	Ensure <code>server.address</code> and <code>server.port</code> resolve	These fields must contain the actual database endpoint, not <code>localhost</code> or <code>127.0.0.1</code> when the DB is remote	Recommended	Validation

2. ActiveGate Extensions

#	Best Practice	Recommended Setting/Value	Priority	Category
7	Build new database integrations on Extensions 2.0	Extensions 2.0 is the current extensions framework; Extensions Framework 1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31); JMX and PMI EF1.0 are deprecated but supported past that date on request. The SQL / Prometheus / SNMP data sources these extensions use run on an Environment (host-based) ActiveGate , the working path; SQL extensions (Oracle, SQL Server, PostgreSQL, MySQL/MariaDB, SAP HANA, Db2, Generic JDBC) can alternatively run on SQL Extension Executor pods in Kubernetes via Dynatrace Operator 1.8+, from Dynatrace 1.346 — a staged rollout, so verify it has reached your tenant; until then the Environment ActiveGate remains the working path	Critical	Deployment
7b	Extensions are YAML-defined with optional Python data sources	YAML package executed by EEC; built-in SQL/Prometheus/SNMP data sources cover most cases; Python data source is for custom logic only — not required by default	Recommended	Architecture
7c	Route extension logs to the <code>default_database_monitoring</code> bucket	Default destination for extension log output; reference this bucket name in IAM policies for DB-team access scoping (see ORGNZ-02 + IAM-04/05)	Recommended	IAM/Bucket

#	Best Practice	Recommended Setting/Value	Priority	Category
8	Install PostgreSQL extension	Captures: connections, transactions/sec, tuple operations, table/index sizes, lock waits, replication lag	Recommended	Extension
9	Install MySQL extension	Captures: connections, queries/sec, buffer pool hit ratio, InnoDB row ops, slow queries	Recommended	Extension
10	Install MS SQL Server extension	Captures: batch requests/sec, page life expectancy, buffer cache hit ratio, deadlocks, wait stats	Recommended	Extension
11	Install Oracle extension	Captures: sessions, tablespace usage, SGA/PGA memory, redo log waits, library cache hit ratio	Recommended	Extension
12	Install MongoDB extension	Captures: connections, operations/sec, document metrics, replica set health, storage engine stats	Recommended	Extension
13	Set extension polling interval to 60 seconds	Default polling; do not exceed 300s or real-time visibility degrades	Recommended	Configuration

Dynatrace Databases App

#	Best Practice	Recommended Setting/Value	Priority	Category
13b	Use the Database App Health Score for first-pass triage	available since 08/2026 (no GA label is applied by any source) for PostgreSQL and MySQL (additional technologies planned); each instance gets a 0-100 score combining availability, performance, configuration, and resource usage — check it before running manual DQL analysis	Recommended	Triage
13c	Use the app's execution-plan visualization as a first pass, not a replacement	Normalizes execution plans across PostgreSQL, MySQL, and SQL Server; fall back to the ActiveGate extension metrics above (rows 8-11) and the DQL patterns in DBMON-02/05 for deep-dive analysis the app's UI doesn't cover	Recommended	Diagnostics

Sources: [Extensions \(DT docs\)](#) — the Extensions 2.0 framework and the EEC execution model, [Extension data sources \(DT docs\)](#) — the built-in SQL / Prometheus / SNMP data sources and the optional Python data source, [Run SQL extensions on Kubernetes \(DT docs\)](#), [Enable Dynatrace SQL database extensions \(DT docs\)](#), [What's new in Dynatrace SaaS 1.337 \(DT docs\)](#) — the `default_database_monitoring` bucket. **Derived:** the EF 1.0 end-of-support dates are carried from the extensions lifecycle announcements rather than a single page — re-confirm before planning a migration around them.

3. Span Attribute Coverage

#	Best Practice	Recommended Setting/Value	Priority	Category
14	Validate all six core DB span attributes	Every database span must carry: <code>db.system</code> , <code>db.query.text</code> , <code>db.operation.name</code> , <code>db.namespace</code> , <code>server.address</code> , <code>server.port</code>	Critical	Data Quality
15	Check for NULL fields periodically	Run filter <code>isNotNull(db.operation.name)</code> — NULL can mean the instrumentation carries the command in another field (<code>code.function</code> for OneAgent MongoDB, <code>span.name</code> for OTel Redis) rather than a driver fault; check those before investigating the driver version	Recommended	Data Quality
16	Verify <code>span.kind</code> is client for outgoing DB calls	All database spans from the calling application must have <code>span.kind == "client"</code> (lowercase)	Recommended	Data Quality
17	For Kafka, verify <code>messaging.system</code> attribute	Kafka spans use <code>messaging.system</code> (not <code>db.system</code>); confirm <code>messaging.destination.name</code> , <code>messaging.operation.type</code> , and <code>messaging.consumer.group.name</code> are populated	Critical	Data Quality
18	For RabbitMQ, verify <code>messaging.system</code> attribute	RabbitMQ spans use <code>messaging.system</code> (not <code>db.system</code>); confirm <code>messaging.destination.name</code> is populated	Critical	Data Quality

4. SQL Database Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
19	Set slow query threshold for SQL databases	500ms — any SQL query exceeding <code>duration</code> > <code>500ms</code> is classified as slow	Critical	Threshold
20	Track read/write ratio	Classify operations: <code>SELECT</code> = READ, <code>INSERT/UPDATE/DELETE</code> = WRITE; monitor ratio shifts over time	Recommended	Analysis
21	Rank queries by total execution time, not average	Sort by <code>total_time_ms = sum(duration)</code> to find highest-impact patterns regardless of individual speed	Critical	Optimization
22	Monitor response time distribution using latency tiers	Bucket into: <code><1ms</code> , <code>1-10ms</code> , <code>10-100ms</code> , <code>100ms-1s</code> , <code>>1s</code> — track tier distribution over time	Recommended	Analysis
23	Baseline P50, P95, P99 over 24 hours	Run hourly percentile query on <code>duration</code> for each <code>db.system</code> ; this is your performance reference point	Critical	Baseline
24	Monitor vendor-specific patterns	PostgreSQL: vacuum/lock contention; MySQL: buffer pool/replication lag; MSSQL: page life expectancy/deadlocks; Oracle: tablespace/SGA	Recommended	Vendor
25	Track errors by the recorded exception	Group <code>span.status_code == "error"</code> spans by the exception recorded in <code>span.events</code> (<code>exception.type</code> , <code>exception.message</code>) to distinguish connection timeouts, pool exhaustion, deadlocks, and constraint violations; <code>span.status_message</code> is experimental and often empty	Critical	Error Handling

Sprint-1.338 Update (May 2026)

#	Best Practice	Recommended Setting/Value	Priority	Category
25b	Use DQL <code>timeseries</code> for table/index metrics — classic Metrics API v2 path is blocked	The <code>postgres.tables.*</code> , <code>postgres.indexes.*</code> , <code>mysql.tables.*</code> , <code>mysql.indexes.*</code> metric keys are blocked from the classic Metrics API v2 path as of sprint-1.338 (May 2026). Query via DQL <code>timeseries</code> against Grail instead.	Critical	API/Migration

In community practice these thresholds are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.

5. NoSQL Database Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
26	Set slow query threshold for NoSQL databases	100ms for MongoDB; 300ms critical threshold for all NoSQL	Critical	Threshold
27	Monitor MongoDB by collection	Group by <code>db.collection.name</code> in addition to <code>db.namespace</code> — collection-level granularity reveals hot spots	Critical	MongoDB
28	Track DynamoDB Scan-to-Query ratio	<code>Scan</code> operations read every item in the table; ratio must be Query-dominated; any high Scan count signals missing Global Secondary Indexes	Critical	DynamoDB
29	Set Cassandra slow threshold at 200ms	<code>filter duration > 200ms</code> — Cassandra operations exceeding 200ms indicate partition hotspots or cross-DC reads	Recommended	Cassandra
30	Monitor Cosmos DB by operation and error rate	Track <code>ReadItem</code> , <code>CreateItem</code> , <code>Query</code> , <code>ReplaceItem</code> separately; errors indicate RU throttling (HTTP 429)	Recommended	Cosmos DB
31	Classify NoSQL read/write operations correctly	READ: <code>find</code> , <code>Query</code> , <code>GetItem</code> , <code>SELECT</code> , <code>ReadItem</code> , <code>get</code> , <code>search</code> ; everything else: WRITE	Recommended	Analysis
32	Run cross-database comparison	Compare <code>total_calls</code> , <code>avg_ms</code> , <code>p95_ms</code> , <code>error_rate_pct</code> across all NoSQL systems in a single query for unified health view	Recommended	Analysis

6. Cache Monitoring (Redis / Memcached)

#	Best Practice	Recommended Setting/Value	Priority	Category
33	Set Redis slow command threshold at 5ms	<code>filter duration > 5ms</code> — Redis operations should complete in microseconds; anything over 5ms is abnormal	Critical	Threshold
34	Monitor Redis read/write balance	READ: <code>GET</code> , <code>MGET</code> , <code>HGET</code> , <code>HGETALL</code> , <code>LRANGE</code> , <code>SMEMBERS</code> , <code>ZRANGE</code> ; WRITE: <code>SET</code> , <code>MSET</code> , <code>HSET</code> , <code>LPUSH</code> , <code>RPUSH</code> , <code>SADD</code> , <code>ZADD</code> , <code>DEL</code>	Recommended	Analysis
35	Alert on Redis latency spikes	Track <code>p95_us</code> (microseconds) over time at 5m intervals; any sustained increase above 5000us (5ms) triggers investigation	Critical	Alerting
36	Watch for expensive Redis commands	<code>KEYS *</code> , <code>SMEMBERS</code> on large sets, <code>LRANGE</code> with large ranges cause latency spikes — flag any occurrence	Critical	Anti-Pattern
37	Report cache metrics in microseconds, not milliseconds	Redis and Memcached latency is sub-millisecond; use <code>avg(duration) / 1us</code> for microsecond precision (dividing a duration by a plain number leaves it a duration)	Recommended	Reporting

.NET Redis Client Coverage — OneAgent 1.337 and 1.343

#	Best Practice	Recommended Setting/Value	Priority	Category
37b	Confirm StackExchange.Redis instrumentation on .NET workloads	OneAgent 1.337 added StackExchange.Redis coverage for .NET applications. Verify your .NET services produce <code>db.system == "redis"</code> spans.	Recommended	Coverage

#	Best Practice	Recommended Setting/Value	Priority	Category
37c	Confirm ServiceStack.Redis instrumentation on .NET workloads	Forthcoming / rolling out with OneAgent 1.343 (released 07/28/2026): tracing support added for ServiceStack.Redis in .NET applications. Rollout is per agent fleet, not per tenant — verify the OneAgent version installed on the hosts concerned (the tenant version is not the agent version), then confirm those services produce <code>db.system == "redis"</code> spans. On OneAgent 1.342 and earlier only StackExchange.Redis is auto-instrumented, and ServiceStack.Redis calls produce no database spans	Recommended	Coverage

In community practice these thresholds are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.

7. Message Broker Monitoring (Kafka / RabbitMQ)

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Track Kafka throughput by topic	Group by <code>messaging.destination.name</code> and <code>messaging.operation.type</code> (<code>publish</code> vs <code>process</code>) at 5m intervals	Critical	Kafka
39	Monitor Kafka consumer group processing latency	Group by <code>messaging.consumer.group.name</code> and <code>messaging.destination.name</code> ; track <code>avg_ms</code> and <code>p95_ms</code> per group	Critical	Kafka
40	Detect Kafka consumer errors	Filter <code>span.status_code == "error"</code> on <code>messaging.operation.type == "process"</code> spans; any sustained errors indicate poisoned messages or deserialization failures	Critical	Kafka
41	Track RabbitMQ publish/consume rate per queue	Group by <code>messaging.destination.name</code> and <code>messaging.operation.type</code> ; a publish rate exceeding consume rate signals backpressure	Critical	RabbitMQ
42	Monitor RabbitMQ message throughput trend	Use <code>makeTimeseries</code> at 5m intervals grouped by <code>messaging.operation.type</code> to detect throughput drops	Recommended	RabbitMQ

8. Search Engine Monitoring (Elasticsearch)

#	Best Practice	Recommended Setting/Value	Priority	Category
43	Set Elasticsearch slow query threshold at 200ms	<code>filter duration > 200ms</code> for search operations; indexing operations use a separate threshold	Recommended	Threshold
44	Break down Elasticsearch operations by type	Track <code>db.operation.name</code> (search, index, bulk, delete) separately; search latency and indexing throughput have different SLOs	Recommended	Analysis
45	Include OpenSearch in all Elasticsearch queries	Always filter <code>in(db.system, {"elasticsearch", "opensearch"})</code> — treat both identically	Recommended	Compatibility

9. Query Analysis and Optimization

#	Best Practice	Recommended Setting/Value	Priority	Category
46	Detect N+1 queries	Group spans by <code>trace.id</code> and <code>db.query.text</code> ; any pattern with <code>calls_per_trace >= 10</code> is an N+1 candidate; <code>>= 50</code> is confirmed N+1	Critical	Anti-Pattern
47	Fix N+1 by batching	Replace N individual queries with a single batch query using <code>IN</code> clauses or JOINS	Critical	Remediation
48	Detect missing indexes heuristically	SELECT queries with <code>call_count >= 50</code> AND <code>avg_ms > 10</code> AND high <code>stddev_ms / avg_ms</code> ratio are index candidates	Recommended	Optimization
49	Detect queries getting slower over time	Run 24h <code>makeTimeseries</code> of <code>avg(duration)</code> by <code>db.system</code> at 1h intervals; rising trend signals index degradation or table growth	Recommended	Trend
50	Flag dynamic SQL / poor parameterization	Count one-off query patterns (<code>call_count == 1</code>); high count per <code>db.system</code> means queries are not parameterized, preventing plan caching	Recommended	Anti-Pattern
51	Prioritize optimization by total time impact	Sort query patterns by <code>total_time_ms = sum(duration)</code> , not by <code>avg_ms</code> — a fast query called 1M times has more impact than a slow query called once	Critical	Optimization

#	Best Practice	Recommended Setting/Value	Priority	Category
52	Use the tail ratio to detect outliers	Calculate <code>p99_ms / p50_ms</code> ; a ratio above 10 indicates severe tail latency requiring investigation	Recommended	Analysis
53	Monitor connection pool pressure	Track <code>countIf(span.status_code == "error")</code> grouped by <code>dt.entity.service</code> and <code>server.address</code> ; connection refused or timeout errors indicate pool exhaustion	Critical	Connection Pool
54	Track error rate trend over 6h at 5m intervals	Use <code>makeTimeseries</code> with <code>total</code> and <code>errors</code> counts; a rising error trend within 30 minutes demands immediate action	Critical	Error Handling

10. Dashboards and KPIs

#	Best Practice	Recommended Setting/Value	Priority	Category
55	Build a unified health overview tile	Single query: group by <code>db.system</code> , show <code>total_calls</code> , <code>avg_ms</code> , <code>p95_ms</code> , <code>error_rate_pct</code> , <code>slow_rate_pct</code>	Critical	Dashboard
56	Include a "total database calls" single-value tile	<code>fetch spans, from:-1h \ filter isNotNull(db.system) \ summarize total_db_calls = count()</code>	Critical	Dashboard
57	Add "top 10 heaviest services" table tile	Group by <code>dt.entity.service</code> with <code>call_count</code> , <code>avg_ms</code> , <code>error_count</code> ; resolve names with <code>entityName()</code>	Critical	Dashboard
58	Show P50 vs P95 vs P99 trend chart	6h timeseries at 5m intervals; three lines on one chart reveals tail latency divergence	Critical	Dashboard
59	Add read vs write ratio trend tile	Classify operations into READ/WRITE; 6h timeseries at 5m intervals	Recommended	Dashboard
60	Add queries-per-minute trend by db.system	6h timeseries at 1m intervals; used for throughput anomaly detection	Recommended	Dashboard
61	Add today-vs-yesterday volume comparison	Use <code>append</code> pattern: <code>from:-24h</code> for today, <code>from:-48h, to:-24h</code> for yesterday	Recommended	Dashboard
62	Include slow query detail table	Last 15m, <code>duration > 500ms</code> , fields: <code>start_time</code> , <code>db.system</code> , <code>db.namespace</code> , <code>db.query.text</code> , <code>duration_ms</code> , <code>service_name</code>	Critical	Dashboard

11. Alerting Thresholds

Error Rate Alerts

#	Best Practice	Recommended Setting/Value	Priority	Category
63	Alert on error rate > 5% for 5 minutes	Severity: Critical — immediate page	Critical	Alert
64	Alert on error rate > 1% for 15 minutes	Severity: Warning — notification	Critical	Alert
65	Alert on any connection refused error	Severity: Critical — immediate page; zero tolerance for connection failures	Critical	Alert
66	Alert on any deadlock detection	Severity: Warning — notification	Recommended	Alert

Slow Query Alerts (P95 Thresholds)

#	Best Practice	Recommended Setting/Value	Priority	Category
67	SQL Warning threshold	P95 > 200ms over 5-minute window	Critical	Alert
68	SQL Critical threshold	P95 > 500ms over 5-minute window	Critical	Alert
69	NoSQL Warning threshold	P95 > 100ms over 5-minute window	Critical	Alert
70	NoSQL Critical threshold	P95 > 300ms over 5-minute window	Critical	Alert
71	Cache (Redis/Memcached) Warning threshold	P95 > 5ms over 5-minute window	Critical	Alert
72	Cache (Redis/Memcached) Critical threshold	P95 > 20ms over 5-minute window	Critical	Alert
73	Elasticsearch Warning threshold	P95 > 500ms over 5-minute window	Recommended	Alert
74	Elasticsearch Critical threshold	P95 > 2000ms over 5-minute window	Recommended	Alert

Throughput Alerts

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Alert on throughput drop > 50% vs 7-day average	Compare current queries/min to 7-day baseline; sudden drop indicates upstream failure or connectivity loss	Critical	Alert
76	Alert on slow query count > 10 per 5-minute window	Sustained slow query volume indicates systemic degradation, not a one-off outlier	Recommended	Alert

12. SLO Definitions

#	Best Practice	Recommended Setting/Value	Priority	Category
77	Define availability SLO	99.9% of database calls succeed, measured over a 24h rolling window as <code>count()</code> - <code>countIf(span.status_code == "error")</code> — <code>span.status_code</code> is null on successful spans, so <code>a != "error"</code> comparison counts nothing	Critical	SLO
78	Define latency SLO	95% of calls complete under the vendor-specific P95 threshold (SQL: 500ms, NoSQL: 300ms, Cache: 20ms, Search: 2000ms)	Critical	SLO
79	Define throughput SLO	No more than 50% drop from 7-day baseline queries/min	Recommended	SLO
80	Measure SLO compliance hourly	Run <code>makeTimeseries</code> at 1h intervals with <code>total</code> and <code>errors / under_threshold</code> counts per <code>db.system</code>	Critical	SLO
81	Track SLO budget burn rate	If 24h availability drops below 99.9%, calculate remaining error budget: $(1 - 0.999) * total_calls - error_count$	Recommended	SLO

13. DQL Query Standards

#	Best Practice	Recommended Setting/Value	Priority	Category
82	Always specify a time range on fetch	<code>fetch spans, from:-1h</code> — never omit <code>from:</code> on span queries; default 2h scan is excessive	Critical	DQL
83	Filter on <code>isNotNull(db.system)</code> first	This is the primary filter to isolate database spans; apply immediately after <code>fetch</code>	Critical	DQL
84	Use named parameters in functions	<code>round(value, decimals: 2)</code> , <code>if(cond, then: "x", else: "y")</code> — positional parameters cause errors	Critical	DQL
85	Alias all aggregations	<code>summarize c = count()</code> , not <code>summarize count()</code> — unaliased aggregations cannot be used in <code>sort</code> or <code>fieldsAdd</code>	Critical	DQL
86	Use duration arithmetic (NOT nanosecond constants)	Write <code>duration / 1ms</code> , <code>duration / 1s</code> , <code>(t2-t1) / 1m</code> — divide a duration by another duration to get a unitless number. Never <code>duration / 1000000.0</code> ; never <code>toLong(duration) / 1ms</code> (<code>long/duration</code> fails type-check). Per <code>dt-dql-essentials</code> .	Critical	DQL
87	Use <code>entityName()</code> (or <code>getNodeName()</code> on <code>smartscape</code>) to resolve service IDs	<code>fieldsAdd service_name = entityName(dt.entity.service, type:"dt.entity.service")</code> for the legacy form; <code>fieldsAdd service_name = getNodeName(dt.smartscape.service)</code> for the modern Smartscape form. Never show raw entity IDs in dashboards.	Recommended	DQL
88	Filter early, sort last	Apply <code>filter</code> immediately after <code>fetch</code> ; apply <code>sort</code> only after <code>summarize</code> ; never <code>sort</code> right after <code>fetch</code>	Critical	DQL
89	Use <code>countIf()</code> for conditional aggregation	<code>errors = countIf(span.status_code == "error")</code> inside <code>summarize</code> — do not use separate filter + count	Recommended	DQL

#	Best Practice	Recommended Setting/Value	Priority	Category
90	Calculate error rate with safe division	<code>error_rate_pct = round((toDouble(error_count) / toDouble(total_calls)) * 100, decimals: 2)</code> — use <code>toDouble()</code> to avoid integer truncation	Recommended	DQL

Modern DQL standards (added 2026-05-07)

#	Best Practice	Recommended Setting/Value	Priority	Category
91	Use <code>event.status</code> / <code>event.start</code> / <code>event.end</code> on <code>dt.davis.problems</code>	<code>NOT status</code> / <code>end_time</code> (recurring authoring bug). Status values are "ACTIVE" and "CLOSED" (not "OPEN"). For duration: <code>(event.end - event.start) / 1m</code> (NOT <code>event.end - timestamp</code> — <code>timestamp</code> is record-write time, not problem-open time).	Critical	DQL
92	Prefer <code>smartscapeNodes</code> "<TYPE>" for new entity queries	Modern Smartscape topology query: <code>smartscapeNodes "SERVICE"</code> , <code>smartscapeNodes "HOST"</code> , etc. Legacy <code>fetch dt.entity.<type></code> still works on hybrid tenants but is deprecated.	Recommended	DQL
93	Wrap multiple <code>summarize</code> aggregations in <code>{}</code>	When summarizing 2+ aggregations, wrap them: <code>summarize { a = count(), b = avg(x) }, by: {field}</code> . Tenant emits <code>PARAMETERS_SHOULD_BE_GROUPED</code> INFO if not wrapped (query still runs but is non-canonical).	Recommended	DQL

14. Where to Go Deeper — Topic Series Map

DBMON covers database monitoring foundations. These topic series cover adjacent domains in depth:

Domain	Topic Series	Notebooks
Distributed tracing & spans (DB spans live here)	SPANS	9
OpenPipeline log processing	OPLOGS	9
OpenPipeline beyond logs (metrics from spans)	OPIPE	7
Dynatrace Intelligence (Davis anomaly detection on DB metrics)	AIOPS	8
Workflows & alert notifications (DB alert routing)	WFLOW	12

Domain	Topic Series	Notebooks
Dashboard strategy & executive reporting	DASH	8
Bucket strategy (<code>default_database_monitoring</code> and custom DB buckets)	ORGNZ	11
IAM administration (bucket-scoped policies for DB-team access)	IAM	15
Tagging strategy (host-group + ownership tagging for DBs)	FAQ-01, FAQ-02	2+
Configuration automation (Monaco/Terraform for extension deployment)	AUTOM	14
OpenTelemetry integration (OTel-instrumented DBs)	OTEL	9
Cloud DB integrations (RDS, Aurora, Cosmos DB, Cloud SQL)	CLOUD	9

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.